LABORATORY RECORD

Experiment 4: SQL Agent with Tool Use

(ReAct Framework)

Submitted By:

D. megha vardhan

Roll No: 2311cs040180

Department of Computer Science & Engineering
Malla Reddy University, Hyderabad

1. Objective & System Architecture

The objective of this experiment is to design and implement a ReAct-based (Reasoning and Acting) SQL agent. Unlike static Text-to-SQL workflows, a ReAct agent operates in iterative cycles: thinking about the task, executing dynamic database tools to gather schema or row insights, observing tool output, and then formulating the next action. This allows the model to troubleshoot query errors, list tables, and analyze table schemas dynamically before running a final safe read-only SQL query.

The ReAct Reasoning & Execution Cycle

```
[User Query] -> Thought -> Action -> Observation -> [Loop] -> Final Answer
```

2. E-Commerce Database Schema & Tools

The agent has access to a local SQLite database (ecommerce.db) containing three tables:
1. customers: Stores customer names, emails, countries, and signup dates.
2. products: Stores product inventory, pricing, and category classifications.
3. orders: Transaction log mapping customer purchases to product quantities.

The agent utilizes three specific database tools:
- list_tables: No arguments. Returns all table names in the database.
- get_schema: Argument: table_name. Returns table DDL details.
- execute_query: Argument: SQL query. Runs read-only SELECT queries.

3. Agent Execution Trace

User Question: 'What is the total revenue of products bought by Alice Smith?'

```
Thought: I need to find the tables in the database to locate where customer details and order details are
stored.
Action: list_tables
Action Input: None
------------------------------------------------------------
Observation: ['customers', 'products', 'orders']
------------------------------------------------------------
Thought: I see the tables: 'customers', 'products', and 'orders'. I need to look at the schema of the
'customers' table to find how to filter by name.
Action: get_schema
Action Input: customers
------------------------------------------------------------
Observation: CREATE TABLE customers (
        customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        country TEXT NOT NULL,
        signup_date TEXT NOT NULL
    )
------------------------------------------------------------
Thought: Now I need the schema for the 'orders' table to see how it relates to customers and how total
amount/revenue is stored.
Action: get_schema
Action Input: orders
------------------------------------------------------------
Observation: CREATE TABLE orders (
        order_id INTEGER PRIMARY KEY AUTOINCREMENT,
        customer_id INTEGER NOT NULL,
        product_id INTEGER NOT NULL,
        order_date TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        total_amount REAL NOT NULL,
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
        FOREIGN KEY (product_id) REFERENCES products(product_id)
    )
------------------------------------------------------------
Thought: I see that 'customers' has 'customer_id' and 'name', and 'orders' has 'customer_id' and
'total_amount'. I can join 'customers' and 'orders' on 'customer_id' and filter by customer name 'Alice Smith'
to sum the 'total_amount'.
Action: execute_query
Action Input: SELECT SUM(o.total_amount) FROM orders o JOIN customers c ON o.customer_id = c.customer_id WHERE
c.name = 'Alice Smith';
------------------------------------------------------------
Observation: Columns: ['SUM(o.total_amount)']
Rows: [(1500.0,)]
------------------------------------------------------------
Thought: I now have the final answer. The sum of total amount for orders by Alice Smith is 1500.0.
Final Answer: The total revenue of products bought by Alice Smith is $1,500.00.
------------------------------------------------------------
```

Synthesized Final Answer

The total revenue of products bought by Alice Smith is $1,500.00.